

Auditoría de Chronos Fútbol — versión 30 de septiembre

Producto: Chronos Fútbol — aplicación web/PWA para control de tiempos, datos, informes y comunicación en fútbol base.

Archivo analizado: APPCRONOSFÚTBOL30SEPT.zip .

Criterio: valorar la aplicación como si fuera a publicarse mañana para clubes, entrenadores, familias y jugadores menores.

Fecha de auditoría: 2 de octubre de 2026.

Veredicto ejecutivo

La aplicación ha avanzado mucho en funcionalidad y en controles internos. El build ya es real, hay una batería extensa de pruebas, App Check está planteado en modo `enforce` , las invitaciones nuevas utilizan tokens opacos y existen reglas de Firebase bastante más cuidadas que en la primera versión.

Pero la entrega del 30 de septiembre introduce o conserva **dos bloqueantes graves para un lanzamiento público inmediato**:

1. El ZIP contiene una carpeta `backups/` con cinco volcados de datos reales, incluido un export de cuentas de Auth y respaldos de usuarios, partidos en directo y solicitudes.
2. La auditoría de dependencias vuelve a detectar **vulnerabilidades altas**: una en `@grpc/grpc-js` del árbol principal y varias asociadas a `nodemailer` en Cloud Functions. Ambas tienen corrección disponible según `npm audit` .

Además, la suite no está completamente verde y el comando de lint del proyecto sigue sin poder ejecutarse desde el paquete tal como se entrega.

Puntuación para sacar la aplicación al público mañana: $\frac{6}{10}$.

Puntuación del producto funcional, aislando la higiene de la entrega y los bloqueantes de seguridad: aproximadamente $\frac{8}{10}$.

No recomendaría un lanzamiento público abierto mañana. Sí recomendaría continuar con una beta técnica cerrada, pero únicamente después de retirar los respaldos del artefacto, rotar cualquier credencial o sesión potencialmente afectada, actualizar dependencias y repetir la auditoría desde una instalación limpia.

Controles ejecutados

Control	Resultado	Lectura
Build	306/306 validaciones del build pasan	<code>npm run build</code> termina con <code>BUILD OK</code> ; revisa sintaxis, HTML, Service Workers, backend, JSON, recursos, precache, PWA y sellos de versión.
Pruebas automatizadas	306/307 pasan	Solo falla <code>test_copias_seguridad.js</code> , en la comprobación de que el artefacto no contiene backups.
Lint mediante <code>npm run lint</code>	FALLA	El binario <code>node_modules/.bin/eslint</code> dentro del ZIP tiene permisos <code>0644</code> y devuelve <code>Permission denied</code> .
Lint ejecutado directamente	0 errores, 114 warnings	La configuración <code>eslint.config.js</code> ya existe y funciona; quedan advertencias de variables no usadas, bloques vacíos y autoasignaciones.
Dependencias raíz	1 vulnerabilidad alta	<code>@grpc/grpc-js@1.14.4</code> , con advisory de validación de certificados y corrección disponible.
Dependencias Functions	1 vulnerabilidad alta agregada por varias advisories	<code>nodemailer@9.1.1</code> , con advisories de exposición de credenciales/DNS y denegación de servicio en parser.
App Check	Código en enforce	El cliente inicializa reCAPTCHA v3 en hosts registrados y las callable sensibles invocan el verificador. Falta acreditar el despliegue real.
Invitaciones nuevas	Mejoradas	Token opaco, caducidad de 14 días, consumo único y reglas de actualización restringida.
Artefacto distribuible	NO APTO	Incluye datos operativos y potencialmente personales dentro de <code>backups/</code> .

Puntuación por diez áreas

Nº	Área evaluada	Nota	Valoración
1	Propuesta de valor	9/10	Resuelve un problema claro del fútbol base y tiene diferenciación comercial.
2	Funcionalidad deportiva	8,5/10	Cronómetro, minutos, sustituciones, asistencia, convocatorias, partidos, informes, directo y planificación.
3	Fiabilidad de tiempos y datos	8,5/10	Los tests de reloj, sincronización, persistencia, replay y semáforos pasan. Falta una prueba de campo prolongada en dispositivos reales.
4	UX y uso durante el partido	7,5/10	Hay controles responsive y para tablet; el volumen funcional puede resultar denso bajo presión.
5	Calidad y regresiones	8/10	306 pruebas pasan y solo queda una roja, pero una suite de producción no debería cerrarse en rojo.
6	Build y entrega	7/10	Build sólido, pero el ZIP contiene datos que nunca deberían viajar y <code>npm run lint</code> está roto por permisos.
7	Autenticación y autorización	8/10	Roles, clubes, revocación, multirol y reglas están cubiertos por muchos tests. Falta una prueba de penetración y validación de Firebase desplegado.
8	Seguridad antipiratería y antiabuso	6/10	App Check y reglas mejoran el control del abuso; el cliente web sigue siendo inspeccionable y los respaldos expuestos constituyen un fallo grave de higiene de seguridad.
9	Protección de datos de menores	5,5/10	La política y los avisos médicos han mejorado, pero el propio ZIP expone volcados con datos y continúa pendiente resolver formalmente el modelo responsable/encargado.
10	Operación, copias y producción	4,5/10	Existe runbook y código de backups, pero se están distribuyendo respaldos dentro del paquete y la comprobación de copias sigue roja.

Resultado final para el lanzamiento de mañana: $\frac{6}{10}$.

Hallazgo crítico: respaldos sensibles dentro del ZIP

La entrega contiene una carpeta `backups/` con cinco ficheros, aproximadamente 671 KB en total:

- `auth_users_2026-09-22-22-11-56.json` — export de cuentas de Firebase Auth;
- `backup_users_...antes_reparacion_v758.json` — respaldo de usuarios;
- `respaldo_live_matches_16082026.json` — respaldo de partidos en directo;
- `respaldo_solicitudes_zombi_...json` — respaldo de solicitudes;
- `respaldo_users_antes_limpieza_...json` — respaldo de usuarios.

El propio código documenta que estos backups pueden contener correos, identificadores, datos de jugadores menores e información de autenticación o hashes. Que `.gitignore` incluya `backups/` **no soluciona el problema del ZIP**: la exclusión de Git impide el seguimiento normal, pero no impide que una herramienta de empaquetado copie la carpeta.

Esto es un bloqueante directo de protección de datos y de seguridad de la cadena de entrega. Antes de compartir o desplegar la aplicación hay que:

1. retirar `backups/` del ZIP y de cualquier artefacto de hosting;
2. comprobar que los backups tampoco aparecen en el directorio público ni en el precache del Service Worker;
3. borrar copias compartidas accidentalmente donde proceda;
4. valorar rotación de credenciales, tokens o sesiones si el export contiene información utilizable;
5. revisar los registros de distribución del ZIP y notificar internamente el incidente si los datos ya llegaron a terceros;
6. hacer que el pipeline falle si encuentra `backups/`, `CAPTURAS/`, `.env`, exports o ficheros de credenciales en el artefacto final.

Hasta completar este punto, **no debe publicarse ni entregarse el paquete**.

Dependencias con vulnerabilidades altas

La auditoría de producción de npm detecta:

@grpc/grpc-js@1.14.4

Afecta al árbol principal a través de Firebase Admin/Google Cloud. El advisory señala que, en ciertas configuraciones, `getAuthContext` puede aceptar certificados no autorizados como si fueran válidos. `npm audit` indica que existe una actualización disponible.

nodemailer@9.1.1

Es dependencia directa de Cloud Functions y se utiliza para el envío de invitaciones. `npm audit` detecta advisories que incluyen:

- reutilización de información TLS/servidor entre transportes con posible exposición de credenciales en ciertos escenarios;
- límites de parser y posibles denegaciones de servicio;
- parsing de destinatarios con comportamientos incorrectos o costosos.

La actualización no debe hacerse a ciegas: hay que actualizar `functions/package.json` y `lockfile`, ejecutar todas las pruebas, probar el envío de invitaciones con destinatarios válidos e inválidos y desplegar primero en staging. Pero la situación actual **no es compatible con una afirmación de $10/10$ en seguridad**.

Seguridad antipiratería y protocolos

Controles positivos comprobados

- App Check con reCAPTCHA v3 en hosts definidos.
- Verificación de App Check en callable Functions sensibles mediante `APPCHECK_MODE = 'enforce'`.
- Autorización por rol y club en Firestore.
- Reglas de invitación que separan `get` de `list`, impiden enumerar invitaciones y limitan la actualización a `updatedAt / usedBy`.
- Tokens de invitación nuevos sin correo, rol ni nombre de club en la URL.
- Cabeceras de seguridad en Firebase Hosting: CSP, HSTS, `X-Frame-Options`, `Referrer-Policy` y `Permissions-Policy`.
- Secretos de correo tratados mediante Secret Manager/variables de entorno, no mediante una clave visible en el cliente.
- Auditoría y registros de acciones administrativas.

Límites reales de la antipiratería

Una PWA JavaScript no puede impedir completamente la copia del código del cliente. El código se descarga al navegador y puede ser inspeccionado. La protección eficaz debe centrarse en que una copia del frontend no pueda leer ni modificar datos:

- reglas Firebase cerradas;
- App Check;
- identidad y claims server-side;
- revocación y expiración de sesiones;
- límites de frecuencia;
- alertas de abuso;
- secretos fuera del cliente;
- control de licencias, marca y derechos de autor.

La aplicación está bien encaminada en estos controles, pero el hecho de empaquetar respaldos reales demuestra que la **seguridad del proceso de entrega** todavía no está al mismo nivel que la seguridad lógica del código.

Protección de datos y menores

La política pública contiene mejoras relevantes: identifica al titular, explica Firestore en `eur3`, reconoce que Functions opera en `us-central1`, describe invitaciones de 14 días, borrado/anonimización, derechos y el riesgo del texto libre en informes.

También se ha corregido la redacción médica: no se promete falsamente que no existe ningún dato relacionado con lesiones; se distingue entre una marca de lesión y un diagnóstico. El formulario incluye un aviso para no introducir diagnósticos o datos médicos en informes.

Sin embargo, siguen existiendo tres cuestiones importantes:

1. **Backups expuestos:** es el problema más grave de esta entrega, porque convierte una protección de producción en una filtración por distribución.
2. **Responsable/encargado:** `privacy.html` mantiene una atribución de roles jurídicos que la documentación técnica considera pendiente de confirmar. El club normalmente decide qué jugadores registra y para qué, mientras la plataforma podría operar por cuenta del club; debe resolverlo un asesor y reflejarse en política y contrato.
3. **Texto libre:** el aviso reduce el riesgo, pero no impide totalmente introducir datos de salud de un menor. Deben existir reglas de uso, procedimiento de rectificación/supresión,

retención adecuada y, si procede, una evaluación de impacto.

La ausencia de DNI, domicilio, fotografía o teléfono del menor es positiva, pero no convierte por sí sola el tratamiento en seguro o conforme: nombres/alias, dorsales, asistencia, informes, lesiones, vínculos familiares y comunicaciones siguen siendo datos personales.

Invitaciones y PII en URLs

El flujo nuevo genera URLs con `?invite=<token>` y guarda el correo y el rol en Firestore, no en la URL. Las reglas exigen caducidad y consumo único.

No obstante, permanecen referencias de compatibilidad con enlaces antiguos basados en `register=true`, `email`, `role` y `clubName`. Eso no afecta a los enlaces nuevos, pero los enlaces antiguos pueden seguir dejando PII en historiales, logs, capturas o chats. Debe definirse una fecha de revocación, migración o caducidad de esos enlaces antes de afirmar que la exposición está totalmente resuelta.

Build, tests y lint

El build actual es una mejora clara y pasa todas sus comprobaciones. La batería ha crecido a 307 tests y el resultado es $306/307$.

El único test rojo es `test_copias_seguridad.js`, concretamente la comprobación de que el artefacto no contiene ficheros de `backups/`. En esta versión ya no se trata de un falso positivo del ZIP: **el test ha detectado correctamente que los ficheros sensibles están realmente dentro de la entrega.**

El lint directo termina con 0 errores y 114 warnings, lo que indica que la configuración nueva funciona conceptualmente. Sin embargo, el comando oficial `npm run lint` falla porque el ZIP incluye `node_modules/.bin/eslint` sin permiso de ejecución (0644). La solución profesional es no distribuir `node_modules` en el ZIP y reconstruirlo con `npm ci` en CI, o corregir la preservación de permisos y añadir un pipeline limpio.

Qué debe corregirse antes de publicar

Prioridad bloqueante

1. Eliminar inmediatamente `backups/` del paquete y del hosting.
2. Revisar el posible incidente de datos y rotar credenciales/tokens si aplica.

3. **Actualizar** `@grpc/grpc-js` y `nodemailer` a versiones no afectadas; repetir `npm audit`.
4. **Conseguir** `npm test` **completamente verde**, incluida la prueba de que ningún artefacto contiene backups o secretos.
5. **Hacer que** `npm run lint` **funcione desde una instalación limpia** con `npm ci` y configuración reproducible.
6. **Ejecutar despliegue staging y pruebas end-to-end** con App Check real, dos clubes aislados y datos ficticios.

Prioridad alta

1. Revocar o caducar enlaces antiguos con PII.
2. Confirmar jurídicamente responsable/encargado, contrato de tratamiento, base legal y proceso de consentimiento de familias.
3. Completar simulacro de restauración de Firestore y Auth sin reutilizar volcados dentro del producto.
4. Añadir una regla de pipeline que bloquee `.env`, `backups/`, `CAPTURAS/`, exports JSON y secretos en cualquier entrega.
5. Probar móvil/tablet y partido completo con pérdida de red, reinicio y recuperación.
6. Reducir los 114 warnings de lint o justificar los que sean deliberados.

Decisión final

Como idea y producto funcional: $\frac{8}{10}$.

Como código local con build y pruebas: $\frac{8}{10}$, aunque queda un test rojo.

Como seguridad antipiratería y antiabuso: $\frac{6}{10}$ por la combinación de backups expuestos y dependencias vulnerables.

Como protección de datos para menores en la entrega actual: $5,\frac{5}{10}$.

Como aplicación para abrir al público mañana: $\frac{6}{10}$.

Veredicto final: la aplicación tiene una base funcional sólida y varias correcciones importantes, pero la entrega del 30 de septiembre **no está lista para un lanzamiento público mañana**. El ZIP contiene datos sensibles reales y las dependencias incluyen vulnerabilidades altas con corrección disponible. Si se retiran los respaldos, se revisa el posible incidente, se actualizan las dependencias, se deja la suite completamente verde y se valida el despliegue real, la puntuación podría volver a subir a **8,5–9/10**. El $\frac{10}{10}$ exigiría

además una revisión de seguridad independiente, cumplimiento de protección de datos validado jurídicamente y operación estable demostrada.